ISTVQA: A Diagnostic Framework for Enabling Indic Scene-Text VQA in Small Vision-Language Models

Anonymous Author(s)

Abstract

Vision-language models (VLMs) perform well on English scene-text visual question answering (VQA) but degrade sharply on Indic scripts such as Bengali, Hindi, and Tamil. Prior work addresses this by proposing new models, architectural changes or datasets, but none systematically diagnoses *which architectural component*—tokenizer, vision encoder, connector, or language model—constitutes the binding bottleneck for each script. We introduce IstVQA, a diagnostic framework that isolates per-component failures through a systematic methodology. We begin by selecting three open-weight small-scale base VLMs—SmolVLM2 (2B), Qwen2.5-VL (3B), and PaliGemma2 (3B)—spanning diverse connector architectures, and apply a tokenizer fertility gate that eliminates SmolVLM2 due to severe Indic fragmentation. The two surviving models then undergo a controlled ablation grid: two training configurations (LLM LoRA-only versus LLM LoRA with connector unfreeze), three dataset scales ($\mathcal{D}_1$=8.7K, $\mathcal{D}_2$=20.7K, $\mathcal{D}_3$=40.2K samples), across four languages, yielding 12 fine-tuning experiments. Our Case A/B classification reveals that Qwen2.5-VL's Indic failures are predominantly Language-side (Case A): LoRA alone lifts the overall judge score from 0.581 to 0.789 (35.8%), while connector unfreezing provides no benefit. PaliGemma2 exhibits a mixed profile: LoRA-only improves from 0.507 to 0.707, but connector unfreezing additionally helps at larger data scales (0.720 at $\mathcal{D}_3$), indicating a partial Vision-Projection bottleneck (Case B). We open-source IstVQA-qa, a multilingual Indic scene-text VQA training dataset at three scales, and IST-VQA-Bench[1], a curated evaluation benchmark of ∼400 samples across Bengali, Hindi, Tamil, and English, along with our complete evaluation pipeline. Our results provide practitioners with a concrete diagnostic methodology for identifying and resolving script-specific VQA failures in small VLMs.

CCS Concepts

• **Computing methodologies** → **Computer vision**; *Natural language processing*.

Keywords

Vision-language models, scene-text VQA, Indic languages, diagnostic framework, LoRA fine-tuning

ACM Reference Format:

Anonymous Author(s). 2026. ISTVQA: A Diagnostic Framework for Enabling Indic Scene-Text VQA in Small Vision-Language Models. In *Proceedings of 17th Indian Conference on Computer Vision, Graphics and Image Processing (ICVGIP2026)*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Recent vision-language models (VLMs) such as SmolVLM2 [15], Qwen2.5-VL [1], PaliGemma2 [17], and commercial systems like Gemini, ChatGPT have demonstrated strong performance on scene-text visual question answering (VQA) for English. These models can read text embedded in natural images—signboards, product labels, documents—and answer questions about it with high accuracy. However, this capability degrades dramatically when the scene text is in Indic scripts such as Bengali, Devanagari (Hindi), or Tamil.

Prior work on Indic-language vision-NLP falls into two categories. The first proposes new datasets [7, 14] that document the problem but do not diagnose its source. The second proposes new models or architectural changes [3, 16] . Neither approach provides the practitioner with a *methodology*—a systematic procedure for determining, given a specific VLM and a target script, where the bottleneck lies and what to do about it.

We address this gap with IstVQA, a diagnostic framework built around a controlled ablation grid. Our key insight is that two simple training configurations—LoRA on the LLM only versus LoRA on the LLM with full connector unfreeze—can discriminate between two failure modes: Language-side bottleneck and Vision-Projection bottleneck.

By crossing these two configurations with three dataset scales ($\mathcal{D}_1$, $\mathcal{D}_2$, $\mathcal{D}_3$) and two architecturally distinct VLMs, we obtain a diagnostic matrix that reveals per-script, per-model bottleneck locations.

Our contributions are:

(1) A **diagnostic framework** (IstVQA) with Case A/B classification that identifies per-script architectural bottlenecks in VLMs, providing practitioners with an actionable methodology rather than a black-box improvement.

(2) **IstVQA-qa and IST-VQA-Bench**, an open-source curated VQA training corpus and evaluation benchmark for Indic scene-text VQA covering Bengali, Hindi, Tamil, and English across six data sources (outdoor text, signboards, documents, posters, crawled images, synthetic text etc.).

[1] https://github.com/soumya997/IST-VQA-Bench

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ICVGIP2026, December 21–25, 2026, Kolkata, India

© 2026 Copyright held by the owner/author(s).

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM.
https://doi.org/10.1145/nnnnnnn.nnnnnnn

(3) **Empirical findings** from 12 fine-tuning experiments across two models, showing that Qwen2.5-VL is Case A (Language-side bottleneck; LoRA alone gives 35.8%) while PaliGemma2 exhibits mixed Case A/B behavior (connector unfreezing helps at scale). We additionally quantify English capability regression, finding that Qwen2.5-VL is remarkably stable while PaliGemma2 suffers moderate degradation.

2 Related Work

Scene-Text VQA Early VQA models could answer questions about images but struggled when answers required reading text within the scene. Two concurrent works addressed this gap: TextVQA [16] introduced 45K questions over 28K Open Images samples and proposed LoRRA (Look, Read, Reason & Answer)—the first model to explicitly incorporate an OCR module alongside image and text encoders, using a copy mechanism that extended the fixed vocabulary with detected OCR tokens. ST-VQA [3] contributed a complementary dataset (31K questions, 23K images from multiple sources) and the ANLS metric, which softly penalizes OCR errors instead of requiring exact match.

From Layout-Aware Models to VLMs. The 2022–2023 period saw a shift toward layout-aware pretraining and generative architectures. LaTr [2] injected spatial positions of OCR tokens into a T5 encoder-decoder pretrained on document images, while GIT [21] showed that a simple generative image-to-text model scaled to large data could match specialized pipelines. Being generative, these models could produce arbitrary answers without vocabulary constraints. By 2023, general-purpose VLMs subsumed scene-text VQA entirely: PaLI and PaLI-X [4, 5] achieved state-of-the-art on TextVQA and ST-VQA as part of broad VLM training, and subsequent models—InternVL [6], Molmo [9]—natively handle text reading without external OCR, while exhibiting additional capabilities such as object detection, counting, and spatial pointing.

Multilingual Scene-Text VQA and Indic Resources. MUST-VQA [19] was the first to explicitly tackle multilingual scene-text VQA, employing an external OCR system (Rosetta-en or Microsoft-OCR) to detect and transcribe text tokens in the TextVQA pipeline. However, its approach remained English-centric in design. The multilingual gap became more visible with MTVQA [18], the first benchmark with human-annotated multilingual text-centric VQA across 9 languages. For Indic languages specifically, IndicSTR12 [14] and BSTD [7] provided scene-text recognition and detection datasets, and IndicVisionBench [13] introduced Indic VQA evaluation across 16 languages.

3 Background: Models and Tokenizers

We study three VLMs spanning different architectural choices. All share a common pipeline: a vision encoder (ViT) extracts patch features from the input image, a connector projects these features into the language model's embedding space, and the LLM generates the answer conditioned on both visual and textual tokens.

SmolVLM2-2.2B-Instruct [15]. Uses a SigLIP ViT with an MLP connector feeding into the SmolLM2 language model. We use this as a *failure-taxonomy reference*—the smallest model in our study, expected to exhibit the most pronounced failures across all scripts. Its tokenizer is GPT-2-based BPE with a small vocabulary that fragments Indic text heavily.

Qwen2.5-VL-3B-Instruct [1]. Employs a ViT with NaViT-style dynamic resolution [8], a 2-layer MLP connector, and the Qwen2.5 LLM. Its byte-level BPE tokenizer has a 151K vocabulary with decent Indic coverage, representing a middle ground in tokenizer fertility.

PaliGemma2-3B-mix-448 [17]. Combines a SigLIP ViT with a single linear-layer connector and the Gemma 2 LLM [10]. Its 256K SentencePiece vocabulary provides the best Indic fertility among the three models. This creates a key tension: PaliGemma2 has the strongest tokenizer for Indic scripts, but if it still fails at Indic scene-text VQA, the bottleneck must be upstream in the ViT or connector—precisely the kind of diagnosis our framework is designed to make.

4 ISTVQA-qa, IST-VQA-Bench: Dataset Construction

We construct IstVQA-qa, a dataset for Indic scene-text VQA, at three scales: $\mathcal{D}_1$ (8.7K samples), $\mathcal{D}_2$ (20.7K samples), and $\mathcal{D}_3$ (40.2K samples). Each scale is a strict superset of the previous one. We describe the data sources, annotation pipeline, and quality controls below.

4.1 Data Sources

Our images come from six sources spanning diverse scene-text conditions:

BSTD [7]: A large-scale dataset for Indian language scene text recognition, consisting of 6,582 scene images of signboards, storefronts, and public notices featuring 1,26,292 words in 11 Indian languages and English. We filter images containing Bengali, Hindi, Tamil, and English text, providing naturalistic scene text in cluttered backgrounds.

Docs: A heterogeneous collection of document images including children's book pages, examination papers, handwritten notes, and posters. These images contain dense Indic text in structured layouts.

Crawl: Web-crawled images containing Indic text from diverse online sources, filtered for quality and relevance.

IndicSTR12 [14]: Cropped word-level images from the IndicSTR12 scene-text recognition dataset, providing coverage of multiple Indic scripts in natural settings.

Synth: Synthetically generated images with Indic text rendered in varied fonts, colors, and backgrounds, used to augment underrepresented scripts and styles.

TextVQA [16]: English scene-text VQA images from the TextVQA benchmark, included to maintain English capability and provide a performance ceiling reference.

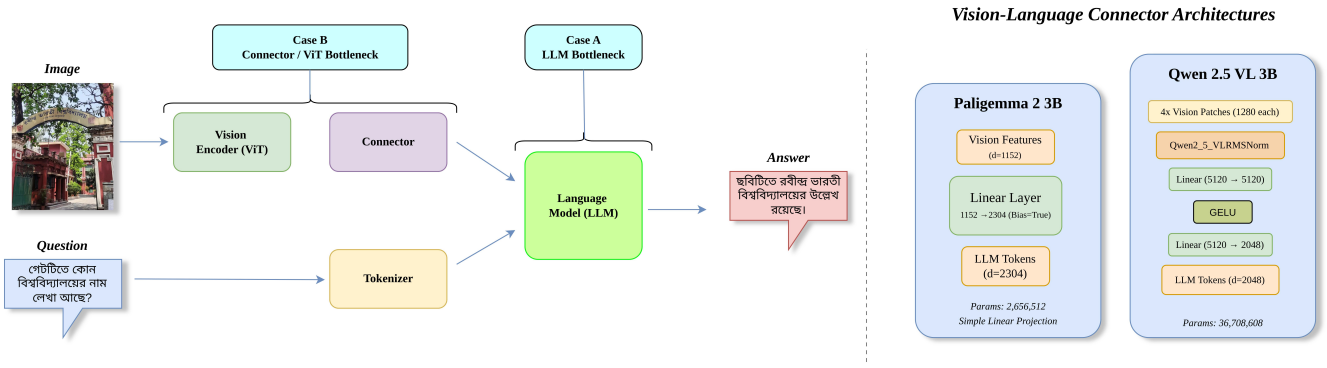


Figure 1: Left: general VLM architecture with our two training configurations. Config 1 applies LoRA (r=16) to the LLM only. Config 2 additionally unfreezes the connector. Right: connector architectures for PaliGemma2 (single linear layer, 2.7M params) and Qwen2.5-VL (2-layer MLP with GELU, 36.7M params).

4.2 QA Generation Pipeline

We generate question-answer pairs using a Gemini-based annotation pipeline. For each image, the pipeline: (1) identifies visible text regions and their content, (2) generates up to five question-answer pairs that require reading the scene text to answer, and (3) ensures questions span different reasoning types (direct reading, inference, spatial reasoning). Generated pairs are verified through Label Studio annotation with human quality checks.

4.3 Dataset Splits and Quality Control

Splits are performed at the *image level*: all QA pairs from a single image appear in the same split (train, validation, or test), preventing information leakage. For sources where five QA pairs are generated per image (BSTD, Docs), this constraint is critical.

We apply MD5 hash deduplication to remove duplicate images that arise from sequential filenames across different collection rounds (e.g., `00000.jpg` in different Crawl runs pulls different images sharing the same filename). The IST-VQA-BENCH evaluation split comprises 393 samples balanced across languages. Table 1 summarizes the dataset composition across scales and sources.

To monitor whether Indic fine-tuning degrades general English visual understanding, we construct a Eng-VQA-Bench of 300 samples: 100 each from VQAv2 (validation set), PixMo-AMA, and PixMo-Count. For PixMo-AMA and PixMo-Count we use the training splits, as AMA provides only a training parquet and Count validation contains empty point annotations.

5 Diagnostic Framework

5.1 The VLM Pipeline

A VLM processes a scene-text VQA query through four stages:

(1) **Tokenizer:** Encodes the text query into token IDs. Poor Indic fertility increases sequence length and may fragment morphologically meaningful units.

Table 1: IstVQA-qa, IST-VQA-Bench dataset statistics by scale and language.

Language	$\mathcal{D}_1$	$\mathcal{D}_2$	$\mathcal{D}_3$	Test
Bengali	2,161	5,700	10,129	97
Hindi	2,177	5,151	10,162	99
Tamil	2,175	5,169	9,793	97
English	2,175	5,175	10,175	100
Total	**8,688**	**20,672**	**40,259**	**393**

(2) **Vision Encoder (ViT):** Extracts patch-level features from the image. Must recognize Indic glyphs as distinct visual patterns.

(3) **Connector:** Projects ViT features into the LLM's embedding space. A weak connector may discard or distort script-specific visual information.

(4) **Language Model (LLM):** Generates the answer conditioned on projected visual features and tokenized query. Requires script-specific grounding.

5.2 Ablation Design

Step 0: Tokenizer Fertility Gate. Before diagnosing structural bottlenecks, we must ensure the LLM can efficiently process the script. We measure tokenizer fertility (tokens/character). If a model exhibits severe fragmentation (e.g., SmolVLM2 requires >1.7 tokens/char for Bengali/Tamil), the tokenizer is the primary bottleneck and structural ablations will be uninformative. Such models are eliminated (Table 2).

We then define two training configurations that probe different pipeline stages:

Config 1: LoRA-only on LLM. We apply low-rank adaptation [12] (rank r = 16) to the LLM's attention layers while keeping the ViT and connector frozen. This tests whether the LLM alone is the bottleneck—if LoRA-only training yields substantial improvement, the upstream stages (ViT and connector) are passing Indic features correctly, and the LLM simply needs script grounding.

Config 2: LoRA on LLM + full connector unfreeze. In addition to LLM LoRA, we fully unfreeze the connector weights. This tests whether the connector is an upstream bottleneck—if this configuration outperforms Config 1, the connector was distorting or discarding Indic features that the ViT successfully extracted.

We do not apply LoRA to the connector because the connectors in our target models are very small: PaliGemma2 uses a single linear layer, and Qwen2.5-VL uses a 2-layer MLP. Full unfreeze is the appropriate intervention at this scale. We exclude ViT training from our ablation grid. Fine-tuning the vision encoder is computationally expensive (it dominates the parameter count) and risks catastrophic forgetting of general visual features.

5.3 Case A/B Classification

Given the results of both configurations on a specific model–script pair, we classify the failure mode:

Case A (Language-side bottleneck): Config 1 (LoRA-only) shows significant improvement over zero-shot. Referring to Figure 1 (Right), the connector—whether PaliGemma2's single linear layer or Qwen2.5-VL's 2-layer MLP—already projects Indic visual features adequately; the LLM just needs script grounding. This is the cheaper fix—LoRA fine-tuning on a few thousand samples suffices.

Case B (Vision-Projection bottleneck): Config 1 shows limited improvement. If Config 2 helps, the upstream projection path shown in Figure 1 (Right) is the bottleneck—the connector fails to relay Indic features to the LLM. If neither config helps, this *suggests* an upstream ViT-level bottleneck; however, confirming this attribution would require full vision-encoder fine-tuning, which is outside the scope of our study (alternative explanations such as insufficient LoRA capacity or data noise cannot be ruled out). Case B requires more invasive intervention.

5.4 Dataset Scaling as Control

Training at three dataset scales ($\mathcal{D}_1 \rightarrow \mathcal{D}_2 \rightarrow \mathcal{D}_3$) serves as a control variable that distinguishes data-starved from architecture-limited failures. If performance scales monotonically with data, the model has the architectural capacity to handle the script and simply needs more training signal. If performance plateaus or regresses with more data, the bottleneck is structural.

5.5 Training Details

All experiments use the SFTTrainer from the TRL library [20] with LoRA rank $r = 16$ (Prior work [12] has shown $r = 16$ to be sufficient for most fine-tuning tasks), $\alpha = 32$, dropout 0.1, learning rate 1×10^{-4} with cosine scheduling, weight decay 0.05, warmup ratio 0.03, gradient accumulation over 16 steps, and bf16 precision. We train for 1 epoch with early stopping based on validation loss (checkpointing every 100 steps, restoring the best checkpoint). Loss curves from $\mathcal{D}_1$ experiments confirm convergence well before the epoch boundary; additional epochs risk overfitting, especially for the connector-unfreeze configuration that adds only ∼3–4% additional trainable parameters. Training is performed on a single NVIDIA RTX 5090 GPU.

6 Evaluation Protocol

6.1 VLM-as-Judge

Our primary evaluation uses a VLM-as-judge approach with Gemini 3.1 Flash Lite as the judge model. The judge receives the original image, the question, the reference answer, and the model's predicted answer, then scores the prediction on four sub-metrics:

- **Visual grounding** (weight $w = 0.35$): Does the answer demonstrate correct reading of text visible in the image?
- **Question relevance** ($w = 0.25$): Does the answer address what was asked?
- **Reference alignment** ($w = 0.20$): Is the answer consistent with the reference?
- **Language correctness** ($w = 0.20$): Is the answer in the correct script with proper grammar?

Each sub-metric is scored on [0, 1], and the overall score is the weighted sum. Critically, the judge uses the image as the primary source of truth, not the reference answer. This design avoids penalizing valid alternative answers that differ from the reference but correctly read the scene text.

7 Experiments and Results

7.1 Tokenizer Fertility Comparison and Model Elimination

Table 2 reports average tokens per character (tok/char) measured across ∼100 sentences per language. Lower values indicate more efficient encoding. PaliGemma2's 256K SentencePiece vocabulary is dramatically more efficient on Indic scripts: Bengali requires only 0.543 tok/char versus 1.080 for Qwen2.5-VL and 1.824 for SmolVLM2—a 2–3.4× compression advantage. All three models are equally efficient on English (0.200 tok/char). SmolVLM2's Indic fertility is extremely poor across all three scripts (Bengali: 1.824, Tamil: 1.731, Hindi: 1.081), indicating that its GPT-2-derived BPE tokenizer fragments Indic text into near-byte-level sequences. This severe fragmentation makes meaningful Indic fine-tuning impractical at the 2B parameter scale, as the model would need to learn script grounding from excessively long, semantically opaque token sequences. We therefore eliminate SmolVLM2 from further fine-tuning experiments and proceed with Qwen2.5-VL and PaliGemma2 only.

7.2 Experiment Grid

Table 3 lists all fine-tuning experiments. We run 6 experiments on Qwen2.5-VL (E6–E11) and 6 on PaliGemma2 (E12–E17), plus zero-shot baselines for all models.

Table 2: Tokenizer fertility: average tokens per character (tok/char) across ∼100 sentences per language. Lower is better (more efficient encoding).

Language	SmolVLM2	Qwen2.5-VL	PaliGemma2
Bengali	1.824	1.080	**0.543**
Hindi	1.081	0.918	**0.326**
Tamil	1.731	1.088	**0.440**
English	0.200	0.200	0.200

Table 3: Experiment grid: 12 fine-tuning runs across 2 models × 2 configs × 3 dataset scales. SmolVLM2 is eliminated at the tokenizer fertility check.

Exp	Model	Config	Dataset
E6	Qwen2.5-VL	LoRA	$\mathcal{D}_1$ (8.7K)
E7	Qwen2.5-VL	LoRA	$\mathcal{D}_2$ (20.7K)
E8	Qwen2.5-VL	LoRA	$\mathcal{D}_3$ (40.2K)
E9	Qwen2.5-VL	LoRA+Conn	$\mathcal{D}_1$ (8.7K)
E10	Qwen2.5-VL	LoRA+Conn	$\mathcal{D}_2$ (20.7K)
E11	Qwen2.5-VL	LoRA+Conn	$\mathcal{D}_3$ (40.2K)
E12	PaliGemma2	LoRA	$\mathcal{D}_1$ (8.7K)
E13	PaliGemma2	LoRA	$\mathcal{D}_2$ (20.7K)
E14	PaliGemma2	LoRA	$\mathcal{D}_3$ (40.2K)
E15	PaliGemma2	LoRA+Conn	$\mathcal{D}_1$ (8.7K)
E16	PaliGemma2	LoRA+Conn	$\mathcal{D}_2$ (20.7K)
E17	PaliGemma2	LoRA+Conn	$\mathcal{D}_3$ (40.2K)

Table 4: Zero-shot overall judge scores by model and language on IST-VQA-Bench.

Model	Bn	Hi	Ta	En	All
Qwen2.5-VL-3B	0.447	0.555	0.389	0.920	0.581
PaliGemma2-3B	0.300	0.466	0.315	0.934	0.507

7.3 Zero-Shot Baselines

Table 4 shows zero-shot performance across all models and languages. Qwen2.5-VL achieves 0.581 overall, with strong English (0.920) but weak Indic performance (Tamil is lowest at 0.389). PaliGemma2 scores lower overall (0.507) despite having the best Indic tokenizer fertility, with particularly weak Bengali (0.300) and Tamil (0.315) but the highest English score (0.934). The gap between English and Indic scripts ($\Delta > 0.35$ for both models) motivates the diagnostic investigation.

7.4 Fine-Tuning Results: Qwen2.5-VL

Table 5 presents the complete results for Qwen2.5-VL across all configurations and dataset scales.

The results reveal several key patterns. First, any fine-tuning dramatically improves over zero-shot: even the smallest dataset ($\mathcal{D}_1$, 8.7K samples) with LoRA-only lifts the overall score from 0.581 to 0.752 (29.4%). Second, E8 (LoRA-only on $\mathcal{D}_3$) achieves the best overall score of **0.789** (35.8% over zero-shot). Third, connector unfreezing consistently underperforms LoRA-only at $\mathcal{D}_1$ and $\mathcal{D}_3$: the Δ(LoRA − LoRA+Conn) is 0.028 on $\mathcal{D}_1$, −0.010 on $\mathcal{D}_2$, and 0.024 on $\mathcal{D}_3$. This is a clear Case A signal: the connector and ViT are not the bottleneck.

Table 5: Qwen2.5-VL-3B fine-tuning results. All scores from Gemini 3.1 Flash Lite judge. VG = visual grounding, QR = question relevance, RA = reference alignment, LC = language correctness, OVR = overall (weighted). Best per-column in bold.

Exp	Config	VG	QR	RA	LC	OVR
—	Zero-shot	0.504	0.684	0.448	0.757	0.581
E6	LoRA, $\mathcal{D}_1$	0.663	0.878	0.604	0.929	0.752
E7	LoRA, $\mathcal{D}_2$	0.655	0.884	0.603	0.908	0.745
E8	LoRA, $\mathcal{D}_3$	**0.697**	**0.927**	**0.665**	**0.925**	**0.789**
E9	LoRA+Conn, $\mathcal{D}_1$	0.617	0.875	0.574	0.913	0.724
E10	LoRA+Conn, $\mathcal{D}_2$	0.660	0.893	0.619	0.910	0.755
E11	LoRA+Conn, $\mathcal{D}_3$	0.671	0.910	0.628	0.915	0.765

Table 6: Per-language overall scores for Qwen2.5-VL LoRA-only experiments. Δ is absolute improvement over zero-shot.

Exp	Bengali	Hindi	Tamil	English
Zero-shot	0.447	0.555	0.389	0.920
E6 ($\mathcal{D}_1$)	0.719	0.769	0.605	0.909
E7 ($\mathcal{D}_2$)	0.698	0.778	0.601	0.898
E8 ($\mathcal{D}_3$)	0.731	0.819	0.700	0.899
Δ (E8)	+0.284	+0.264	+0.311	−0.021

7.5 Per-Language Analysis: Qwen2.5-VL

Table 6 breaks down the overall judge score by language for the LoRA-only experiments, revealing that all three Indic scripts follow Case A behavior.

Tamil shows the largest relative improvement (79.9%), followed by Bengali (63.5%) and Hindi (47.6%). English remains essentially flat, with a marginal −0.021 decrease that falls within judge variance. This confirms that fine-tuning on IST-VQA-Bench selectively improves Indic performance without degrading English capability.

We observe a non-monotonic scaling pattern between D_1 and D_2 for Qwen2.5-VL (0.752 vs. 0.745). Given that this difference (0.007) is within the margin of training noise, it suggests that within this parameter-efficient fine-tuning regime, simply doubling the data volume provides diminishing returns. However, the subsequent jump to 0.789 at D_3 indicates that once a critical threshold of linguistic diversity is reached, the LoRA adapters successfully capture the required script-specific grounding.

7.6 Fine-Tuning Results: PaliGemma2

Table 7 presents the PaliGemma2 results, which contrast sharply with Qwen2.5-VL.

Table 7: PaliGemma2-3B fine-tuning results on IST-VQA-Bench. All scores from Gemini 3.1 Flash Lite judge. Best per-column in bold.

Exp	Config	VG	QR	RA	LC	OVR
—	Zero-shot	0.426	0.555	0.341	0.837	0.507
E12	LoRA, $\mathcal{D}_1$	0.563	0.830	0.511	0.927	0.678
E13	LoRA, $\mathcal{D}_2$	0.604	0.851	0.531	0.933	0.699
E14	LoRA, $\mathcal{D}_3$	0.587	0.855	0.559	0.926	0.707
E15	LoRA+Conn, $\mathcal{D}_1$	0.556	0.855	0.500	0.917	0.678
E16	LoRA+Conn, $\mathcal{D}_2$	0.605	0.845	0.534	0.916	0.696
E17	LoRA+Conn, $\mathcal{D}_3$	**0.600**	**0.873**	**0.561**	**0.929**	**0.720**

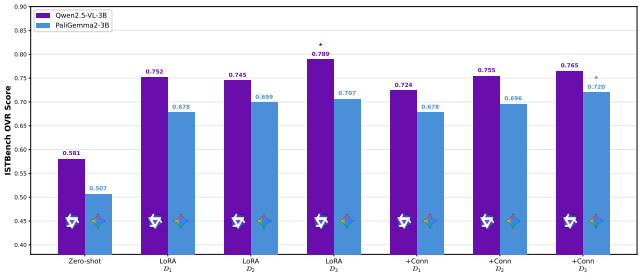


Figure 2: IST-VQA-Bench overall judge scores across all fine-tuning configurations. Qwen2.5-VL peaks with LoRA-only at $\mathcal{D}_3$ (E8, 0.789), while PaliGemma2 peaks with LoRA+Conn at $\mathcal{D}_3$ (E17, 0.720). Stars indicate each model's best checkpoint.

PaliGemma2 shows a different pattern from Qwen2.5-VL. First, the zero-shot baseline is substantially lower (0.507 vs. 0.581), despite PaliGemma2 having superior Indic tokenizer fertility. Second, LoRA-only training lifts performance from 0.507 to 0.707 at $\mathcal{D}_3$ (39.4%), confirming a strong Case A component. Third—and critically—*connector unfreezing additionally helps at the largest scale*: E17 (LoRA+Conn, $\mathcal{D}_3$) achieves 0.720, surpassing E14 (LoRA-only, $\mathcal{D}_3$) by 0.013. At smaller scales the two configurations are tied (both 0.678 at $\mathcal{D}_1$) or LoRA+Conn slightly trails (0.696 vs. 0.699 at $\mathcal{D}_2$). This indicates a *mixed Case A/B* profile: the LLM is the primary bottleneck, but PaliGemma2's single linear-layer connector is a secondary bottleneck that becomes addressable only with sufficient training data.

Figure 2 visualizes the overall IST-VQA-Bench scores for both models across all configurations.

7.7 Per-Language Analysis: PaliGemma2

Table 8 breaks down PaliGemma2 scores by language, highlighting the contrast with Qwen2.5-VL.

Tamil shows the largest absolute gain for PaliGemma2 (0.300), consistent with the Qwen pattern. Bengali improves the least (0.237), suggesting the ViT has particular difficulty with Bengali glyphs. Notably, PaliGemma2's English score *improves* after fine-tuning (0.934 → 0.959), contrasting with the mild degradation seen in Qwen2.5-VL's IST-VQA-Bench English scores.

Table 8: Per-language overall scores for PaliGemma2. LoRA-only (top) and best LoRA+Conn (bottom).

Exp	Bengali	Hindi	Tamil	English
Zero-shot	0.300	0.466	0.315	0.934
E12 ($\mathcal{D}_1$)	0.485	0.723	0.566	0.930
E13 ($\mathcal{D}_2$)	0.520	0.772	0.565	0.931
E14 ($\mathcal{D}_3$)	0.524	0.752	0.591	0.951
E17 (LC, $\mathcal{D}_3$)	0.537	0.759	0.615	0.959
Δ (E17)	+0.237	+0.293	+0.300	+0.025

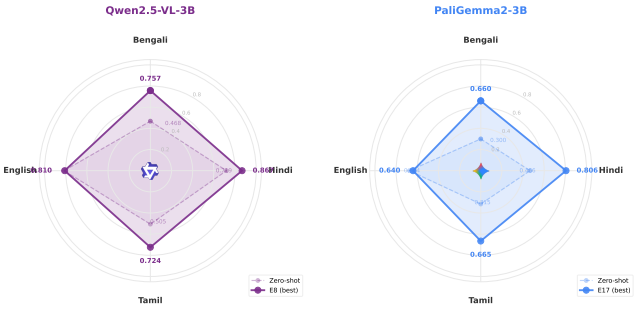


Figure 3: Per-language radar plots for the best checkpoints: Qwen2.5-VL E8 (left) and PaliGemma2 E17 (right). Dashed lines show zero-shot baselines. Both models show large Indic gains with minimal English change; Qwen2.5-VL achieves higher Indic ceilings across all three scripts.

Figure 3 shows the per-language score profiles for the best checkpoint of each model, overlaid with the zero-shot baseline.

7.8 General Capability Regression

To measure whether Indic fine-tuning degrades general English VQA capability, we evaluate all models on Eng-VQA-Bench, a 300-sample English-only benchmark drawn equally from VQAv2 [11], PixMo-Ask-Model-Anything, and PixMo-Count (100 samples each). Table 9 reports aggregate results.

Qwen2.5-VL is stable: LoRA-only training at any scale causes $\leq$ 0.007 regression. Only E11 (LoRA+Conn, $\mathcal{D}_3$) shows a notable drop (−0.043). PaliGemma2 is more sensitive. LoRA-only Eng-VQA-Bench falls with data scale (0.674 → 0.665 → 0.660 → 0.645), a −4.3% relative drop at $\mathcal{D}_3$. However, E15 (LoRA+Conn, $\mathcal{D}_1$) *improves* Eng-VQA-Bench to 0.718 (6.5%)—small-scale connector training helps general feature projection without overfitting to Indic patterns.

Per-source degradation analysis. To understand *how* each model degrades, we decompose Eng-VQA-Bench scores by source for the best checkpoint of each model (Table 9).

Table 9: Per-source Eng-VQA-Bench decomposition for the best checkpoint of each model. Δ% is relative change from zero-shot. Avg. words shows answer length shift after Indic fine-tuning.

Model	Source	OVR	VG	QR	RA	Δ%	Avg. wds
Qwen (ZS)	PixMo-AMA	0.759	0.642	0.940	0.564	–	38.8
	PixMo-Count	0.801	0.711	0.945	0.655	–	9.9
	VQAv2	0.887	0.845	0.980	0.795	–	2.2
Qwen E8 (best)	PixMo-AMA	0.707	0.592	0.875	0.474	−6.9	6.9
	PixMo-Count	0.835	0.755	0.965	0.740	4.2	6.0
	VQAv2	0.888	0.870	0.970	0.810	0.1	1.1
PaliG (ZS)	PixMo-AMA	0.599	0.550	0.685	0.360	–	1.6
	PixMo-Count	0.547	0.407	0.733	0.333	–	1.5
	VQAv2	0.877	0.815	0.985	0.775	–	1.4
PaliG E17 (best)	PixMo-AMA	0.610	0.545	0.705	0.370	1.8	7.0
	PixMo-Count	0.495	0.385	0.625	0.315	−9.5	5.5
	VQAv2	0.815	0.745	0.935	0.720	−7.1	3.0

Table 10: Case A/B diagnosis per model and script. A = Language-side bottleneck (LoRA suffices), A/B = mixed (connector helps at scale).

Model	Bn	Hi	Ta	En
Qwen2.5-VL	A	A	A	—
PaliGemma2	A/B	A	A/B	—

Qwen2.5-VL (E8) retains its core abilities: VQAv2 is unchanged and counting even improves by 4.2%. The only drop is on PixMo-AMA (−6.9%), where average answer length collapses from 38.8 to 6.9 words—the model has learned the short-answer style of the Indic training data, which lowers reference-alignment but does not reflect an actual loss in visual understanding.

PaliGemma2 (E17) degrades across all three sources, with counting hit hardest (−9.5%) and VQAv2 also declining (−7.1%). Only open-ended QA (PixMo-AMA) stays near zero-shot level. Notably, the average answer length also increases substantially after fine-tuning. Unlike Qwen's style-only shift, this reflects genuine capability loss—further supporting the mixed Case A/B diagnosis.

7.9 Case A/B Diagnosis Summary

Table 10 summarizes the Case A/B classification per model and script based on the full ablation results.

For Qwen2.5-VL, all three Indic scripts are clearly Case A: LoRA-only training provides substantial improvement (0.26 to 0.31 absolute), and connector unfreezing does not help. This indicates that Qwen's NaViT-based vision encoder and 2-layer MLP connector successfully extract and project Indic scene-text features; the LLM simply needs script grounding via LoRA.

For PaliGemma2, Hindi is Case A (LoRA-only reaches 0.772; LoRA+Conn 0.759 at $\mathcal{D}_3$—no benefit). Bengali and Tamil are classified as mixed A/B: while LoRA-only provides the majority of the improvement, connector unfreezing at $\mathcal{D}_3$ yields additional gains (Bengali: 0.524 → 0.537; Tamil: 0.591 → 0.615). This suggests PaliGemma2's single linear-layer connector is a secondary bottleneck for these scripts, one that only becomes addressable with sufficient training data to avoid overfitting.

English is classified as "—" (no bottleneck) since zero-shot performance is already high for both models.

7.10 Sub-Metric Analysis

Breaking down by sub-metric reveals where training has the most impact. For Qwen2.5-VL E8 versus zero-shot:

- **Reference alignment** shows the largest absolute gain: 0.448 → 0.665 (0.217), indicating that LoRA teaches the model to produce answers that match expected content.
- **Visual grounding** improves substantially: 0.504 → 0.697 (0.193), confirming better scene-text reading.
- **Question relevance** rises from 0.684 → 0.927 (0.243), the largest absolute gain, showing the model learns to answer what was asked rather than generating generic descriptions.
- **Language correctness** improves from 0.757 → 0.925 (0.168), reflecting better script generation.

The fact that visual grounding improves with LLM-only training (Config 1) is a key diagnostic signal: it suggests the ViT was already extracting scene-text features, but the LLM was not utilizing them effectively.

7.11 Qualitative Examples

Table 11 presents representative examples illustrating each bottleneck type. Row 1 shows a Case A success: Qwen2.5-VL zero-shot hallucinates gibberish for a Bengali comic title, but LoRA fine-tuning corrects it to the accurate answer. Row 2 demonstrates Case A in PaliGemma2, where the model replaces a semantically plausible but visually ungrounded guess with the correct artist name after fine-tuning. Row 3 illustrates Case B: PaliGemma2 cannot localize Hindi text spatially until the connector is unfrozen. Row 4 exposes a ViT-level bottleneck: Qwen2.5-VL produces an identical hallucination before and after fine-tuning on dense handwritten Bengali, indicating the vision encoder itself cannot extract the text.

8 Discussion

Guidelines for Indic VLM adaptation. Our results yield a concrete methodology. The first step is a tokenizer fertility gate: models with severe Indic fragmentation (e.g., SmolVLM2) should be eliminated before any fine-tuning investment. However, passing this gate is necessary but not sufficient—PaliGemma2 has the best Indic fertility among our models (Table 2), yet consistently underperforms Qwen2.5-VL across all scales, indicating that connector architecture and visual grounding matter more than tokenizer quality

Table 11: Qualitative examples illustrating each diagnostic category.

Model	Image Context	User Query (Translation)	Zero-Shot Baseline	Best Fine-Tuned Checkpoint	Diagnostic Takeaway
Qwen2.5-VL	Bengali Comic Book Cover	কমিকটির নাম কী?	পুরুষদের কাছে *(Gibberish)*	বঁটুল দি গ্রেট। *(Correct)*	**Case A:** LoRA successfully resolves severe LLM script hallucination.
Qwen2.5-VL	Hindi Railway Sign	तस्वीर में दिख रहे बोर्ड पर हिंदी में क्या लिखा है?	*unanswerable*	तस्वीर में दिख रहे बोर्ड पर हिंदी में "आनन्द विहार टर्मिनल" लिखा हुआ है। *(Correct)*	**Case A:** LLM fine-tuning enables zero-shot scene-text recognition in native script.
PaliGemma2	Bengali Comic Book Cover	ছবিতে শিল্পীর নাম কী লেখা আছে?	রবীন্দ্রনাথ ঠাকুর। *(Hallucination)*	নারায়ণ দেবনাথ। *(Correct)*	**Case A:** Fine-tuning replaces semantic guessing with visual text grounding.
PaliGemma2	Hindi Railway Sign	तस्वीर में यह हिंदी टेक्स्ट कहाँ स्थित है?	*unanswerable*	हिंदी टेक्स्ट नीले रंग के आयताकार बोर्ड पर... *(Correct)*	**Case B:** Connector unfreezing resolves spatial text localization failures.

alone. For models that pass the gate, begin with LoRA fine-tuning of the LLM alone—this yields substantial gains with negligible English regression. For models with thin connectors (e.g., PaliGemma2's single linear layer), additionally unfreeze the connector once training data exceeds ∼20K samples; below this threshold, connector unfreezing risks overfitting without benefit.

Connector architecture matters. The divergent connector behavior is architecturally informative. Qwen2.5-VL's 2-layer MLP connector has sufficient capacity to learn adequate Indic projections during pretraining, making further unfreezing counterproductive. PaliGemma2's single linear layer, by contrast, acts as a tighter bottleneck that benefits from retraining only when sufficient data prevents overfitting.

English capability tradeoff. The two models exhibit different English regression profiles (Table 9). Qwen2.5-VL is stable: LoRA-only training causes ≤ 0.007 drop across all scales. PaliGemma2 shows moderate degradation reaching -4.3% at $\mathcal{D}_3$, suggesting its LLM has less spare capacity for absorbing new languages without displacing English. This asymmetry provides a diagnostic signal: models showing Eng-VQA-Bench regression may benefit from capacity-preserving techniques such as lower LoRA rank or replay-based regularization.

Limitations. Our study has several limitations that suggest directions for future work. We evaluate only at the 3B scale; larger models may exhibit different bottleneck patterns. Our language coverage (Bengali, Hindi, Tamil, English) spans three Indic scripts but does not capture the full diversity of Indian writing systems. The VLM-as-judge evaluation carries inherent biases, which we mitigate by using a consistent judge across all experiments. We also do not evaluate on established cross-lingual benchmarks such as IndicVisionBench. Future work will explore whether shared images across languages improve multilingual alignment, and whether combining general VQA with scene-text VQA data yields complementary gains.

9 Conclusion

We present IstVQA, a diagnostic framework for identifying per-script architectural bottlenecks in VLMs for Indic scene-text VQA. Through a controlled ablation grid spanning two training configurations, three dataset scales, and two architecturally distinct 3B-parameter VLMs, we show that bottleneck location is architecture-dependent. We initially considered three candidate models—SmolVLM2 2B, Qwen2.5-VL, and PaliGemma2—but eliminated SmolVLM2 based on tokenizer fertility analysis, as its GPT-2-derived BPE tokenizer fragments Indic scripts into near-byte-level sequences (e.g., 1.824 tok/char for Bengali), making meaningful Indic fine-tuning impractical at the 2B scale. Among the remaining two models, Qwen2.5-VL is consistently Case A (Language-side) across all Indic scripts — LoRA alone yields substantial gains with negligible English regression. PaliGemma2 exhibits mixed Case A/B behavior: the language model is the primary bottleneck, but its single linear-layer connector acts as a secondary Vision-Projection bottleneck for Bengali and Tamil, addressable only at larger data scales. Despite superior Indic tokenizer fertility, PaliGemma2 consistently trails Qwen2.5-VL and suffers moderate general-capability degradation — demonstrating that tokenizer quality alone does not determine VQA performance.

Our framework provides practitioners with a concrete methodology: (0) apply the Tokenizer Fertility Gate to eliminate models with severe Indic fragmentation, (1) evaluate zero-shot, (2) apply LoRA to the LLM, (3) if gains plateau or English regresses excessively, unfreeze the connector with sufficient data, (4) if still insufficient, the evidence suggests an upstream ViT bottleneck (full vision-encoder fine-tuning would be needed to confirm). We open-source IstVQA-qa, IST-VQA-Bench, and our complete evaluation pipeline to support future diagnostic studies.

References

[1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2.5-VL: To See the World with Wisdom. In *arXiv*

preprint arXiv:2502.13923.

[2] Ali Furkan Biten, Ron Litman, Yusheng Xie, Srikar Appalaraju, and R. Manmatha. 2022. LaTr: Layout-Aware Transformer for Scene-Text VQA. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 16548–16558.

[3] Ali Furkan Biten, Rubèn Tito, Andrés Mafla, Lluís Gomez, Marçal Rusiñol, Ernest Valveny, C.V. Jawahar, and Dimosthenis Karatzas. 2019. Scene Text Visual Question Answering. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4291–4301.

[4] Xi Chen, Josip Djolonga, Neil Houlsby, Filip Padlewski, Xiaohua Zhai, Ibrahim Alabdulmohsin, Basil Mustafa, Soravit Changpinyo, Wei Wu, Carlos Riquelme Ruiz, et al. 2023. PaLI: A Jointly-Scaled Multilingual Language-Image Model. In *The Eleventh International Conference on Learning Representations (ICLR)*.

[5] Xi Chen, Josip Djolonga, Filip Padlewski, Basil Mustafa, Xiaohua Zhai, Lucas Beyer, Ibrahim Alabdulmohsin, Batchelor Batchelor, Sylvain Gelly, Stefan Heigold, et al. 2023. PaLI-X: On Scaling up a Multilingual Vision and Language Model. *arXiv preprint arXiv:2305.18565* (2023).

[6] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Pan Zhang, Bin Zhu, Lewei Lu, et al. 2024. InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 24185–24203.

[7] Anik De, Abhirama Subramanyam Penamakuri, Rajeev Yadav, Aditya Rathore, Harshiv Shah, Devesh Sharma, Sagar Agarwal, Pravin Kumar, and Anand Mishra. 2025. Bharat Scene Text: A Novel Comprehensive Dataset and Benchmark for Indian Language Scene Text Understanding. *arXiv preprint arXiv:2511.23071* (2025).

[8] Mostafa Dehghani, Basil Mustafa, Josip Djolonga, Jonathan Heek, Matthias Minderer, Mathilde Caron, Andreas Steiner, Joan Puigcerver, Robert Geirhos, Ibrahim Alabdulmohsin, et al. 2024. Patch n' Pack: NaViT, a Vision Transformer for Any Aspect Ratio and Resolution. In *Advances in Neural Information Processing Systems*.

[9] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luke VanderBilt, Onur Walls, Akshita Bhagia, Suvam Biswal, Rohit Burns, Amanpreet Chaudhry, Prafulla Dhariwal, et al. 2024. Molmo and PixMo: Open Weights and Open Data for State-of-the-Art Multimodal Models. *arXiv preprint arXiv:2409.17146* (2024).

[10] Gemma Team. 2024. Gemma 2: Improving Open Language Models at a Practical Size. In *arXiv preprint arXiv:2408.00118*.

[11] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. 2017. Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 6904–6913.

[12] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shuxian Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

[13] Krutrim AI Labs. 2025. IndicVisionBench: A Benchmark for Evaluating Vision-Language Models on Indic Languages.

[14] Arnav Lunia, Ayush Kumar Dash, and Priyanka Advani. 2023. Towards Building an Open-Domain Multilingual Scene Text Recognition System. In *arXiv preprint*.

[15] Andrés Marafioti, Oriane Pannier, Lucile Saulnier, and Thomas Wolf. 2025. SmolVLM – Small Vision Language Models. In *arXiv preprint arXiv:2504.05299*.

[16] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. 2019. Towards VQA Models That Can Read. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8317–8326.

[17] Andreas Steiner, André Susano Pinto, Michael Tschannen, Daniel Keysers, Xiao Wang, Yonatan Bitton, Alexey Gritsenko, Neil Houlsby, Lucas Beyer, and Xiaohua Zhai. 2024. PaliGemma 2: A Family of Versatile VLMs for Transfer. In *arXiv preprint arXiv:2412.03555*.

[18] Jingqun Tang, Qi Qi, Kun Luo, Yongjie Meng, Yue Gu, Chunhui Li, Haibo Chen, Hao Peng, and Chunhui Yuan. 2024. MTVQA: Benchmarking Multilingual Text-Centric Visual Question Answering. In *arXiv preprint arXiv:2405.11985*.

[19] Xirui Tang, Jinghui Lu, Pan Zhang, Zhe Chen, Wenhai Wang, Weijie Su, Lewei Lu, Jiannan Wu, Sen Xing, et al. 2025. MTVQA: A Massive Multilingual Text-centric Visual Question Answering Benchmark. In *Findings of the Association for Computational Linguistics: ACL 2025*.

[20] Leandro von Werra, Edward Beeching, Lewis Tunstall, et al. 2024. TRL: Transformer Reinforcement Learning. HuggingFace TRL library.

[21] Jianfeng Wang, Zhengyuan Yang, Xiaowei Hu, Linjie Li, Kevin Lin, Zhe Gan, Zicheng Liu, Ce Liu, and Lijuan Wang. 2022. GIT: A Generative Image-to-text Transformer for Vision and Language. In *Proceedings of the 30th ACM International Conference on Multimedia*. 149–158.